

RAM CHARAN VANGA

RA2511027010164

Subject : DSA

Unit 3, S3, SLO1 - Stack Implementation Using Linked Lists - Advantages, Disadvantages and Applications

1. In pointer-based stacks, memory is allocated ____.

Answer: dynamically at runtime

2. In pointer-based stacks, there is no need to define the ____ stack size.

Answer: fixed maximum

3. Stack implemented using pointers can easily grow and ____ at runtime.

Answer: shrink

4. There is no ____ of memory since space is allocated as needed.

Answer: wastage

5. Linked list-based stacks can handle a large number of elements until the system runs out of ____.

Answer: heap memory

6. Pointer-based stack avoids ____ overflow, unlike array-based stacks.

Answer: stack

7. In pointer-based implementation, memory is released using ____ when elements are popped.

Answer: free() / deallocation

8. Stacks using pointers are more ____ for programs requiring flexible memory use.

Answer: adaptable / dynamic

9. No need to ____ the stack when it becomes full, unlike arrays.

Answer: reallocate or resize

10. Each node in a linked list-based stack contains both data and a ____ to the next node.

Answer: pointer (next)

11. Stack implementation using pointers requires extra memory for storing ____.

Answer: pointers (addresses)

12. Each node in a linked list-based stack uses more ____ than array-based stacks.

Answer: memory space

13. Accessing elements in a pointer-based stack is ____ than in an array.

Answer: slower (due to pointer dereferencing)

14. Stack operations in linked lists involve dynamic memory allocation, which can be ____.

Answer: time-consuming (overhead)

15. Improper memory management in pointer-based stacks can lead to ____ leaks.

Answer: memory

16. Debugging a pointer-based stack is more ____ than an array-based stack.

Answer: complex

17. Pointer-based stack implementation depends on ____ allocation at runtime.

Answer: dynamic heap

18. Excessive use of pointers increases the risk of ____ errors.

Answer: segmentation fault / dangling pointer

19. It is harder to determine the total number of elements without ____ through the stack.

Answer: traversing

20. Stack operations in pointer-based implementation are not as ____ as array-based stacks for small data sets.

Answer: cache efficient

21. Stack using pointers is useful in handling ____ size data efficiently.

Answer: variable / dynamic

22. Function call management in recursion uses a ____ stack.

Answer: call / execution

23. Pointer-based stacks are suitable for evaluating and converting ____ expressions.

Answer: arithmetic (infix/postfix/prefix)

24. Stacks help in implementing ____ and redo features in applications.

Answer: undo

25. Pointer-based stacks are used in ____ parsing for compilers.

Answer: syntax / expression

26. Backtracking algorithms like maze solving or N-Queens use ____.

Answer: stacks

27. Stack using pointers is helpful in managing ____ frames in programming languages.

Answer: stack / activation

28. Depth-first search (DFS) in graphs can be implemented using a ____.

Answer: stack

29. Stacks are used in checking ____ of parentheses in expressions.

Answer: balance / validity

30. Linked list-based stacks are useful in systems with limited or unpredictable ____ availability.

Answer: memory